



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

Experiment No. : 2

Title: Understanding Data with Programming Exercise



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

Batch: A1 Roll No.: 16014225015 Experiment No.: 2

Aim: Understanding of the Data Handling technique with Programming exercise

Resources needed: Python 3.x environment (Jupyter Notebook / Google Colab etc.), numpy and pandas library

Theory

There are several programming languages that are commonly used for data handling, analysis, and manipulation tasks. Some of the most popular data handling programming languages are Python, R, Scala, Matlab, Julia, and SQL. In this practical session, we will handle Python-based data handling commands. Let us see features of Python.

Python

Python is one of the most widely used programming languages for data handling and analysis. It has rich libraries like Pandas, NumPy, Matplotlib, Seaborn, and SciPy that offer comprehensive support for data manipulation, numerical computing, and data visualization. Python's readability and user-friendly syntax make it a top choice for data scientists, researchers, and analysts. In this session, we will learn some important Python libraries, namely NumPy and Pandas.

NumPy

NumPy, which stands for "Numerical Python," is a fundamental package for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with an extensive collection of mathematical functions to operate on these arrays efficiently. NumPy is the backbone of many other Python libraries for scientific computing and data analysis.

Key features of NumPy:

- **N-dimensional array object (numpy.ndarray):** Efficient data structure for storing and manipulating large datasets, making it suitable for numerical computations and data analysis.
- **Broadcasting:** NumPy automatically broadcasts arrays with different shapes for element-wise operations, reducing the need for explicit loops.
- **Mathematical Functions:** NumPy offers a wide range of mathematical functions like trigonometric, exponential, statistical, and linear algebra functions, among others.
- **Indexing and Slicing:** NumPy supports advanced indexing techniques to access specific elements or subsets of data quickly.
- **Integration with C/C++:** NumPy is written in C and Python, which provides the advantage of speed and efficiency in numerical computations.

Pandas

Pandas is an open-source data manipulation library built on top of NumPy. It provides high-level data structures and functions to make working with structured data easy and intuitive. The two primary data structures in Pandas are the **Series** and **DataFrame**.

Key features of Pandas:

- **Series:** A one-dimensional labeled array, similar to a NumPy array but with additional indexing capabilities, making it more versatile for data analysis.

- **DataFrame:** A two-dimensional labeled data structure with rows and columns, resembling a spreadsheet or SQL table, which is the central data structure for data analysis in Pandas.
- **Data Import/Export:** Pandas can read and write data from various file formats like CSV, Excel, SQL databases, JSON, and HTML tables.
- **Data Exploration:** Pandas provides functions to explore, clean, and preprocess data, such as handling missing values, filtering, grouping, and aggregation.
- **Time-Series Analysis:** Pandas has robust support for handling time-series data, making it popular for financial and temporal data analysis.
- **Data Visualization:** Although not a visualization library, Pandas integrates well with other visualization libraries like Matplotlib and Seaborn, allowing easy creation of basic visualizations directly from DataFrames and Series.

Together, NumPy and Pandas form a powerful combination for data analysis and data manipulation tasks in Python. NumPy handles the underlying numerical computations and efficient data storage, while Pandas provides high-level structures and functions that simplify data handling and analysis tasks.

Exercise: Data Handling Commands with Python

Both Pandas and NumPy offer a wide range of commands and functions for data analysis. Here are some essential commands from each library commonly used in data analysis:

1. Pandas

Creating DataFrames

- `pd.DataFrame(data, columns=['col1', 'col2', ...])`: Creates a DataFrame from data (a dictionary, list of lists, or a NumPy array) with specified column names.

Data Exploration

- `df.head(n)`: Returns the first n rows of the DataFrame.
- `df.info()`: Provides information about the DataFrame, including data types and non-null counts.
- `df.describe()`: Generates summary statistics for numerical columns.
- `df.shape`: Returns the dimensions of the DataFrame (rows, columns).
- `df.columns`: Returns the column names.

Data Selection and Filtering

- `df['column_name']`: Selects a specific column from the DataFrame.
- `df.loc[row_index, 'column_name']`: Accesses specific data using labels for both rows and columns.
- `df.iloc[row_index, col_index]`: Accesses specific data using integer-based indexing.
- `df[df['column_name'] > value]`: Filters data based on a condition.

Data Aggregation

- `df.groupby('column_name').agg({'other_column': 'mean'})`: Grouping and aggregation based on specific columns.
- `df['column_name'].value_counts()`: Calculates the frequency of unique values in a column.

Data Cleaning

- `df.isnull()`: Checks for missing values in the DataFrame.
- `df.dropna()`: Drops rows with missing values.
- `df.fillna(value)`: Fills missing values with a specific value.

Data Visualization

- `df['column_name'].plot(kind='hist')`: Creates a histogram for a specific column.

- `df.plot(x='x_column', y='y_column', kind='scatter')`: Creates a scatter plot.

2. NumPy

Creating Arrays

- `np.array([1, 2, 3])`: Creates a 1-dimensional NumPy array.
- `np.zeros((m, n))`: Creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))`: Creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)`: Creates an array with values from start to stop (exclusive) with a given step size.

Array Operations

- `np.mean(arr)`: Calculates the mean of the array.
- `np.median(arr)`: Calculates the median of the array.
- `np.sum(arr)`: Calculates the sum of the array elements.
- `np.std(arr)`: Calculates the standard deviation of the array.
- `np.unique(arr)`: Returns the unique elements of the array.

Array Indexing and Slicing

- `arr[index]`: Accesses specific elements of the array using indexing.
- `arr[start:stop]`: Slices the array from start to stop (exclusive).

Array Broadcasting

- NumPy automatically broadcasts arrays with different shapes for element-wise operations.

Mathematical Functions

- `np.sin(arr)`, `np.cos(arr)`, `np.exp(arr)`, etc.: Various mathematical functions that operate element-wise on the array

Activity:

- Read dataset related to your chosen AI use case using Numpy and Pandas
- Perform in detail data analysis of this dataset
- Get information and description of dataset.
- See if any null values are present. Display count of null values.
- Choose appropriate technique to handle missing values.(imputation with use of inplace)
- Use sorting of data in dataframe to display topmost 5 or 8 records based on one or more column values(conditional filtering)
- Get frequency listing of any one relevant column(2 cases)
- Sorting of rows and columns,(implicit and explicit indexing)
- Accessing particular row based on certain condition and displaying only one or few columns from it.(3 cases with compound conditions)
- Minimum and maximum values related analysis
- Use of group by on one or more columns(2 cases)
- Adding new column to existing dataframe and populating same using existing columns data.
- Use of appropriate aggregate functions with groupby.(2 cases)

- Selection on particular groups based on name or condition
- Find correlation between any two columns values.
- Try transformation(normalization using any technique) on data set
- Joining, merging and concatenation of data in dataframe.

Write down observation for your dataset for each of above listed task of analysis.

Results: (python script and output and inferences derived)

```
from google.colab import files
import pandas as pd
import numpy as np

uploaded = files.upload()

file_name = next(iter(uploaded))
df = pd.read_csv(file_name)

print("Dataset loaded successfully!")
print("File name:", file_name)
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

display(df.head())
```

*** Choose Files EV_dataset.csv
EV_dataset.csv(text/csv) - 33493889 bytes, last modified: 19/9/2026 - 100% done
Saving EV_dataset.csv to EV_dataset.csv
Dataset loaded successfully!
File name: EV_dataset.csv
Number of rows: 250000
Number of columns: 22

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acc
0	BYD	Model S	Coupe	FWD	Electric	Blue	84.93	464.06	0.85	True	...	Shanghai	7	32213.0	5.394897	
1	Lucid	EV6	Sedan	AWD	Electric	Blue	65.14	345.36	2.03	True	...	Tokyo	7	31125.0	5.067074	
2	Volkswagen	EV6	SUV	FWD	Hybrid	White	22.75	104.25	0.22	True	...	Oslo	5	15674.0	5.057602	
3	BMW	Ioniq	Sedan	AWD	Hybrid	Black	71.72	341.74	0.48	False	...	Vancouver	2	24044.0	4.826029	
4	Volkswagen	Model 3	Truck	AWD	Hybrid	Green	62.99	338.16	0.79	True	...	Shenzhen	4	29567.0	5.222624	

5 rows x 22 columns

```
print("Activity 1")

print("Information and Description of Dataset")
```



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

```
print("\nDataset Information:")

df.info()


print("\nDataset Shape:")

print(df.shape)


print("\nColumn Names:")

print(df.columns.tolist())


print("\nStatistical Description:")

display(df.describe())


print("\nFirst 5 Records:")

display(df.head())


print("\nLast 5 Records:")

display(df.tail())
```

```
Activity 1
Information and Description of Dataset

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 22 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0  manufacturer        250000 non-null object
 1  model               250000 non-null object
 2  type                250000 non-null object
 3  drive_type          250000 non-null object
 4  fuel_type           250000 non-null object
 5  color               250000 non-null object
 6  battery_kwh         250000 non-null float64
 7  range_km            250000 non-null float64
 8  charging_time_hr    250000 non-null float64
 9  fast_charging       250000 non-null bool
10  release_year        250000 non-null int64
11  country             250000 non-null object
12  city                250000 non-null object
13  seats              250000 non-null int64
14  price_usd           250000 non-null float64
15  efficiency_score     250000 non-null float64
16  acceleration_0_100_kmph 250000 non-null float64
17  top_speed_kmph      250000 non-null float64
18  warranty_years       250000 non-null int64
19  cargo_space_liters  250000 non-null float64
20  safety_rating        250000 non-null float64
21  target_high_efficiency 250000 non-null int64
dtypes: bool(1), float64(9), int64(4), object(8)
memory usage: 48.3+ MB

Dataset Shape:
(250000, 22)

Column Names:
['manufacturer', 'model', 'type', 'drive_type', 'fuel_type', 'color', 'battery_kwh', 'range_km', 'charging_time_hr', 'fast_charging', 'release_year', 'country', 'city', 'seats', 'price_usd', 'efficiency_score', 'acceleration_0_100_kmph', 'top_speed_kmph', 'warranty_years', 'cargo_space_liters', 'safety_rating', 'target_high_efficiency']
```

Column Names:
['manufacturer', 'model', 'type', 'drive_type', 'fuel_type', 'color', 'battery_kwh', 'range_km', 'charging_time_hr', 'fast_charging', 'release_year', 'country', 'city', 'seats', 'price_usd', 'efficiency_score', 'acceleration_0_100_kmph', 'top_speed_kmph', 'warranty_years', 'cargo_space_liters', 'safety_rating', 'target_high_efficiency']

Statistical Description:

	battery_kwh	range_km	charging_time_hr	release_year	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty_years	cargo_space_liters	safety_rating	target_high_efficiency
count	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000
mean	75.012960	375.115585	1.075374	2018.511452	4.800124	30515.365240	4.999806	6.249837	199.984872	6.596360	449.937046	4.000117	4.000117
std	20.079595	109.948419	0.672221	4.033158	1.721400	7248.361483	0.764292	2.166595	46.225610	2.419298	99.991551	0.578886	0.578886
min	-18.700000	-79.430000	-0.380000	2012.000000	2.000000	-4565.000000	2.194368	2.500000	120.000000	3.000000	-2.700000	3.000000	3.000000
25%	61.470000	299.120000	0.610000	2015.000000	4.000000	25552.750000	4.448975	4.370000	159.920000	5.000000	382.500000	3.500000	3.500000
50%	74.980000	370.360000	0.860000	2019.000000	5.000000	30502.000000	4.999715	6.250000	199.960000	7.000000	450.200000	4.000000	4.000000
75%	88.550000	446.390000	1.330000	2022.000000	6.000000	35483.000000	5.552174	8.130000	239.970000	8.000000	517.300000	4.500000	4.500000
max	159.070000	872.260000	5.430000	2025.000000	7.000000	60045.000000	8.077820	10.000000	280.000000	10.000000	919.400000	5.000000	5.000000

First 5 Records:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty_years
0	BYD	Model S	Coupe	FWD	Electric	Blue	84.93	464.06	0.85	True	...	Shanghai	7	32213.0	5.394897	5.84	136.00	
1	Lucid	EV6	Sedan	AWD	Electric	Blue	65.14	345.36	2.03	True	...	Tokyo	7	31125.0	5.067074	3.86	149.34	
2	Volkswagen	EV6	SUV	FWD	Hybrid	White	22.75	104.25	0.22	True	...	Oslo	5	15674.0	5.057602	2.85	275.80	
3	BMW	Ioniq	Sedan	AWD	Hybrid	Black	71.72	341.74	0.48	False	...	Vancouver	2	24044.0	4.826029	3.78	130.41	
4	Volkswagen	Model 3	Truck	AWD	Hybrid	Green	62.99	338.16	0.79	True	...	Shenzhen	4	29567.0	5.222624	7.07	253.31	

5 rows x 22 columns

Last 5 Records:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	wa
249995	Lucid	Bolt	Coupe	AWD	Hybrid	Yellow	74.63	419.61	2.36	True	...	Delhi	2	29188.0	5.942839	9.41	215.77	
249996	Chevrolet	Ioniq	Hatchback	AWD	Hybrid	Red	102.25	475.16	0.84	False	...	Paris	7	34595.0	4.846013	5.86	146.19	
249997	Hyundai	i3	Truck	AWD	Hybrid	Red	81.16	400.83	1.08	True	...	Paris	4	29931.0	5.723986	8.95	273.57	
249998	Ford	Mustang Mach-E	Truck	FWD	Hybrid	Silver	67.63	289.30	0.90	True	...	Shanghai	4	25308.0	4.203677	8.27	172.11	
249999	Chevrolet	Mustang Mach-E	Hatchback	AWD	Hybrid	Grey	91.08	540.55	0.76	False	...	Manchester	7	34097.0	5.821790	4.73	127.06	

5 rows x 22 columns

```
print("Activity 2")

print("Checking Null Values")

print("\nNull values in each column:")

print(df.isnull().sum())
```

```
print("\nTotal null values:")

print(df.isnull().sum().sum())

print("\nColumns containing null values:")

print(df.columns[df.isnull().any()].tolist())
```

... Activity 2 Checking Null Values

Null values in each column:

manufacturer	0
model	0
type	0
drive_type	0
fuel_type	0
color	0
battery_kwh	0
range_km	0
charging_time_hr	0
fast_charging	0
release_year	0
country	0
city	0
seats	0
price_usd	0
efficiency_score	0
acceleration_0_100_kmph	0
top_speed_kmph	0
warranty_years	0
cargo_space_liters	0
safety_rating	0
target_high_efficiency	0
dtype: int64	

Total null values:
0

Columns containing null values:
[]



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

```
print("Activity 3")

print("Handling Missing Values")

df.fillna(0, inplace=True)

print("\nNull values after handling:")

print(df.isnull().sum())

print("\nTotal null values after handling:")

print(df.isnull().sum().sum())

print("\nDataset shape:")

print(df.shape)
```

```
... Activity 3
Handling Missing Values

Null values after handling:
manufacturer      0
model              0
type              0
drive_type        0
fuel_type         0
color             0
battery_kwh       0
range_km          0
charging_time_hr  0
fast_charging     0
release_year      0
country           0
city              0
seats             0
price_usd         0
efficiency_score  0
acceleration_0_100_kmph 0
top_speed_kmph    0
warranty_years    0
cargo_space_liters 0
safety_rating     0
target_high_efficiency 0
dtype: int64

Total null values after handling:
0

Dataset shape:
(250000, 22)
```

```
print("Activity 4")

print("Sorting and Displaying Top Records")

print("\nTop 5 EVs by Range:")

display(

    df.sort_values(by='range_km', ascending=False)

    .head(5)
```



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

```
)

print("\nTop 5 EVs by Price:")

display(

    df.sort_values(by='price_usd', ascending=False)

    .head(5)

)

print("\nTop 5 EVs by Battery Capacity:")

display(

    df.sort_values(by='battery_kwh', ascending=False)

    .head(5)

)

print("\nTop 5 EVs by Efficiency Score:")

display(

    df.sort_values(by='efficiency_score', ascending=False)

    .head(5)

)
```

Activity 4
Sorting and Displaying Top Records

Top 5 EVs by Range:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
116106	Chevrolet	i3	Crossover	RWD	Hybrid	Grey	146.96	872.26	3.57	True	...	Vancouver	7	53989.0	5.458604	9.52	137.71	
13889	BYD	Model 3	Hatchback	AWD	Electric	White	146.03	871.73	1.27	False	...	Oslo	7	53238.0	5.716375	6.03	128.49	
110000	Chevrolet	Air	Crossover	FWD	Hybrid	Red	147.46	869.20	1.49	True	...	Austin	5	46793.0	6.729071	8.14	173.28	
130849	Chevrolet	EV6	Sedan	RWD	Hybrid	Blue	147.46	856.81	4.73	False	...	Shenzhen	7	52591.0	5.390797	2.90	136.37	
232283	Volkswagen	Model S	Coupe	FWD	Hybrid	Red	153.13	854.38	1.39	True	...	Pune	4	50545.0	4.690988	5.92	255.87	

5 rows x 22 columns

Top 5 EVs by Price:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
103247	BYD	Mustang Mach-E	Coupe	FWD	Electric	Black	155.65	648.53	1.95	False	...	Tokyo	7	60045.0	3.946649	7.30	239.86	
150798	Hyundai	Model S	SUV	AWD	Electric	Green	151.99	695.24	4.73	True	...	Osaka	5	59458.0	4.905726	8.77	167.12	
32308	BMW	Model X	Hatchback	FWD	Electric	Black	154.24	681.66	1.10	True	...	Delhi	2	59054.0	5.355961	9.69	141.49	
11748	Chevrolet	Bolt	Crossover	FWD	Electric	Black	151.60	671.90	3.04	True	...	Tokyo	2	58695.0	3.550570	6.19	252.46	
20790	Chevrolet	Ionik	Hatchback	AWD	Hybrid	Black	151.77	697.50	1.03	False	...	Manchester	4	58624.0	4.573329	4.93	201.34	

5 rows x 22 columns

Top 5 EVs by Battery Capacity:

Top 5 EVs by Battery Capacity:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
221702	BMW	Model S	Coupe	FWD	Hybrid	Blue	159.07	701.61	3.64	True	...	Berlin	5	55115.0	3.736381	3.99	187.91	
18319	BMW	Air	Truck	AWD	Electric	Black	156.56	803.38	4.89	True	...	Paris	7	52112.0	5.736310	9.11	271.20	
181201	Kia	Ionik	SUV	FWD	Hybrid	Yellow	155.88	653.53	1.11	False	...	Seoul	4	49620.0	3.736196	9.51	209.67	
103247	BYD	Mustang Mach-E	Coupe	FWD	Electric	Black	155.65	648.53	1.95	False	...	Tokyo	7	60045.0	3.946649	7.30	239.86	
94849	Kia	Air	Truck	RWD	Hybrid	Red	155.48	726.38	4.87	False	...	Delhi	7	50594.0	4.878073	8.37	150.84	

5 rows x 22 columns

Top 5 EVs by Efficiency Score:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
107890	Tesla	Ionik	SUV	AWD	Electric	Grey	76.29	456.27	0.57	True	...	Oslo	6	36397.0	8.077820	8.30	221.46	
86272	Ford	Model S	Crossover	FWD	Electric	Green	86.96	496.64	0.80	True	...	Seoul	4	30552.0	7.969830	3.13	216.55	
66448	Volkswagen	Mustang Mach-E	Crossover	RWD	Electric	White	48.96	284.47	0.90	True	...	Seoul	7	20927.0	7.903570	4.92	277.91	
138370	Chevrolet	Leaf	Hatchback	RWD	Hybrid	Yellow	65.86	387.92	0.55	True	...	Seoul	5	24200.0	7.840224	3.10	161.06	
43422	Nissan	EV6	Truck	RWD	Hybrid	Red	37.41	222.93	0.36	False	...	Shenzhen	5	19625.0	7.789375	9.02	268.28	

5 rows x 22 columns

```
print("Activity 5")

print("Frequency Listing")

print("\nFrequency of Manufacturers:")

print(df['manufacturer'].value_counts())

print("\nFrequency of Release Years:")

print(df['release_year'].value_counts().sort_index())
```

... Activity 5 Frequency Listing

Frequency of Manufacturers:

manufacturer

Chevrolet	25314
Kia	25143
Hyundai	25084
Tesla	25067
BMW	25054
Volkswagen	24997
Lucid	24946
BYD	24817
Nissan	24789
Ford	24789

Name: count, dtype: int64

Frequency of Release Years:

release_year

2012	17915
2013	17557
2014	17896
2015	17795
2016	17929
2017	17785
2018	17932
2019	17788
2020	17755
2021	18003
2022	17832
2023	17809
2024	17958
2025	18046

Name: count, dtype: int64

```
print("Activity 6")

print("Indexing and Selecting Data")


print("\nFirst 10 rows:")

display(df.head(10))
```

```
print("\nSelected columns:")

display(df[['manufacturer', 'model', 'battery_kwh', 'range_km',
'price_usd']].head(10))

print("\nRows from index 10 to 20:")

display(df.iloc[10:21])

print("\nEVs with range greater than 500 km:")

display(df.loc[df['range_km'] > 500].head(10))
```

Activity 6
Indexing and Selecting Data

First 10 rows:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
0	BYD	Model S	Coupe	FWD	Electric	Blue	84.93	464.06	0.85	True	...	Shanghai	7	32213.0	5.394897	5.84	136.00	
1	Lucid	EV6	Sedan	AWD	Electric	Blue	65.14	345.36	2.03	True	...	Tokyo	7	31125.0	5.067074	3.86	149.34	
2	Volkswagen	EV6	SUV	FWD	Hybrid	White	22.75	104.25	0.22	True	...	Oslo	5	15674.0	5.057602	2.85	275.80	
3	BMW	Ioniq	Sedan	AWD	Hybrid	Black	71.72	341.74	0.48	False	...	Vancouver	2	24044.0	4.826029	3.78	130.41	
4	Volkswagen	Model 3	Truck	AWD	Hybrid	Green	62.99	338.16	0.79	True	...	Shenzhen	4	29567.0	5.222624	7.07	253.31	
5	BYD	Leaf	Sedan	FWD	Hybrid	Yellow	51.82	285.57	0.66	False	...	Vancouver	4	27351.0	4.846714	9.77	244.02	
6	BYD	ID.4	Coupe	AWD	Electric	Silver	79.37	463.82	2.20	False	...	London	5	36801.0	6.284650	6.41	273.79	
7	BMW	Model S	Hatchback	FWD	Hybrid	Black	88.18	456.20	0.61	True	...	Toronto	6	33785.0	4.608233	8.52	131.93	
8	Chevrolet	Model X	Hatchback	AWD	Electric	Red	61.46	246.52	0.48	False	...	Oslo	4	26174.0	4.316902	7.84	246.43	
9	Volkswagen	Mustang Mach-E	Sedan	AWD	Electric	Silver	78.47	457.47	0.60	True	...	Oslo	6	37608.0	5.590284	4.83	172.03	

10 rows x 22 columns

Selected columns:

	manufacturer	model	battery_kwh	range_km	price_usd
0	BYD	Model S	84.93	464.06	32213.0
1	Lucid	EV6	65.14	345.36	31125.0
2	Volkswagen	EV6	22.75	104.25	15674.0
3	BMW	Ioniq	71.72	341.74	24044.0
4	Volkswagen	Model 3	62.99	338.16	29567.0
5	BYD	Leaf	51.82	285.57	27351.0
6	BYD	ID.4	79.37	463.82	36801.0
7	BMW	Model S	88.18	456.20	33785.0
8	Chevrolet	Model X	61.46	246.52	26174.0
9	Volkswagen	Mustang Mach-E	78.47	457.47	37608.0

Rows from index 10 to 20:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
10	Lucid	Mustang Mach-E	Crossover	FWD	Electric	Black	91.70	453.40	2.30	False	...	Toronto	5	42283.0	5.092665	8.87	235.48	
11	Nissan	Mustang Mach-E	Sedan	RWD	Hybrid	Silver	58.12	315.09	1.50	True	...	London	4	25471.0	4.711994	6.31	265.21	
12	Nissan	Bolt	Crossover	AWD	Hybrid	Yellow	104.56	466.09	3.02	True	...	Vancouver	4	42282.0	4.198497	7.71	260.87	
13	BMW	ID.4	Sedan	AWD	Hybrid	Black	92.85	456.18	1.78	True	...	Paris	5	38410.0	4.961624	8.56	263.37	
14	Lucid	EV6	SUV	RWD	Electric	Red	69.87	339.16	0.55	False	...	Vancouver	7	27825.0	4.670245	4.79	146.34	
15	BYD	Leaf	SUV	AWD	Hybrid	White	133.87	607.61	2.41	True	...	Pune	2	49155.0	4.646276	7.77	178.18	
16	BMW	Model 3	Sedan	FWD	Electric	Blue	36.62	167.34	1.24	False	...	Paris	7	22338.0	4.556377	9.82	185.77	
17	Lucid	ID.4	Truck	AWD	Hybrid	Grey	70.17	376.21	0.77	True	...	Shanghai	6	33020.0	5.932819	4.28	236.51	
18	Ford	EV6	Truck	RWD	Electric	Black	103.10	522.88	2.84	True	...	Toronto	4	40730.0	5.767744	3.63	201.31	
19	BMW	Bolt	Truck	AWD	Hybrid	Silver	83.63	371.36	0.73	False	...	Pune	6	39911.0	4.765055	3.81	230.55	
20	BYD	Model S	Coupe	RWD	Electric	Black	59.33	347.04	0.44	True	...	Oslo	6	32721.0	5.688287	4.21	147.99	

11 rows x 22 columns

Evs with range greater than 500 km:

	manufacturer	model	type	drive_type	fuel_type	color	battery_kwh	range_km	charging_time_hr	fast_charging	...	city	seats	price_usd	efficiency_score	acceleration_0_100_kmph	top_speed_kmph	warranty
15	BYD	Leaf	SUV	AWD	Hybrid	White	133.87	607.61	2.41	True	...	Pune	2	49155.0	4.646276	7.77	178.18	
18	Ford	EV6	Truck	RWD	Electric	Black	103.10	522.88	2.84	True	...	Toronto	4	40730.0	5.767744	3.63	201.31	
24	BYD	ID.4	SUV	RWD	Electric	Black	98.17	520.23	0.75	False	...	Osaka	5	34947.0	4.888936	4.81	206.81	
36	Tesla	Model 3	Crossover	FWD	Hybrid	Green	117.69	573.31	1.01	False	...	Seoul	6	44437.0	4.984781	4.36	176.96	
37	BMW	i3	Sedan	RWD	Hybrid	White	112.44	560.31	1.34	True	...	Oslo	5	48332.0	6.039747	2.86	271.86	
46	Hyundai	Leaf	Hatchback	FWD	Hybrid	Black	111.27	660.64	1.00	True	...	Delhi	4	35334.0	6.139561	7.14	136.18	
55	Lucid	Air	SUV	AWD	Hybrid	Black	105.54	501.68	0.89	True	...	Seoul	5	46190.0	4.696188	5.61	163.75	
75	Nissan	Ioniq	Truck	RWD	Hybrid	Red	99.83	508.85	1.07	False	...	Seoul	6	37990.0	5.600312	4.41	229.04	
97	Chevrolet	Model S	Crossover	RWD	Hybrid	Blue	92.40	537.88	0.90	True	...	Paris	6	32314.0	5.469040	6.93	162.89	
103	Lucid	Model S	Hatchback	AWD	Electric	Red	99.15	539.68	0.98	True	...	Austin	2	32078.0	6.483723	9.53	237.21	

10 rows x 22 columns

Post-Lab Questions:**1. Explain the concept of "Broadcasting" in NumPy with the help of suitable example**

Broadcasting is a feature of NumPy that allows arithmetic operations to be performed between arrays of different shapes without explicitly creating or copying the smaller array.

NumPy automatically expands the smaller array to match the shape of the larger array, provided their dimensions are compatible.

2. Explain the difference between label-based indexing (df.loc[]) and integer-based indexing (df.iloc[]) in Pandas with the help of suitable example.

In Pandas, `df.loc[]` and `df.iloc[]` are used to select specific rows and columns from a DataFrame. The main difference is that `df.loc[]` uses labels, while `df.iloc[]` uses integer positions.

Label-based indexing (`df.loc[]`) selects data using the row or column labels. For example, if a row has the label `B`, `df.loc['B']` selects that particular row.

Integer-based indexing (`df.iloc[]`) selects data according to its numerical position. The indexing starts from 0, so `df.iloc[1]` selects the second row.

For example, if a DataFrame has three rows with labels `A`, `B`, and `C`, then `df.loc['B']` selects the row labelled `B`, whereas `df.iloc[1]` selects the second row.

Therefore, `loc` is label-based indexing, while `iloc` is position-based integer indexing.

Course Outcomes: (refer exp list)

Conclusion: In this topic, we learned the difference between label-based indexing and integer-based indexing in Pandas. We understood that `df.loc[]` is used to access data using labels, while `df.iloc[]` is used to access data using integer positions. Both methods are useful for efficiently selecting specific rows and columns from a DataFrame.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:**Books/ Journals/ Websites:**

1. S.C. Gupta, V. K. Kapoor, *Fundamentals of Mathematical Statistics*, Sultan Chand and Sons, 2020.
2. P. N. Tan, M. Steinbach, Vipin Kumar, *Introduction to Data Mining*, Pearson Education, 2016.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

3. Han, Kamber, *Data Mining Concepts and Techniques*, Morgan Kaufmann, 3rd Edition, 2012
4. NumPy Documentation - <https://numpy.org/doc/>
5. Pandas Documentation - <https://pandas.pydata.org/>

